



ServiceNow ChatOps Connector — Credential Provisioning Guide

Version 0.1.0, 2026-09-02

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3
Reference documentation	3

Overview

This connector creates and reads records in **one** ServiceNow table — a table your organization's ServiceNow ChatOps or Now Actions notification flow is already configured to watch and deliver to Slack or Microsoft Teams. Crogl only writes and reads records in that table; it does not connect to Slack or Teams itself, and does not configure the notification flow that delivers to them.

IMPORTANT

Before doing anything below, confirm with whoever owns your organization's ServiceNow ChatOps / Now Actions setup which table their delivery flow actually watches (commonly **interaction**, **sc_task**, or a custom table), and that it is already live. If no such flow exists yet, this connector will still create records successfully — they simply will not reach anyone in chat. Setting up that delivery flow itself is a separate, one-time ServiceNow configuration task, outside this connector's scope, and is a prerequisite to this guide being useful at all.

These steps require a **ServiceNow admin** (someone who can create OAuth Application Registry entries and assign roles) — not a Crogl role. Your ServiceNow instance must be on the **Washington DC release or later** for inbound OAuth Client Credentials support.

What kind of account to create

ServiceNow's inbound OAuth 2.0 Client Credentials grant type is a machine-to-machine credential mapped to a dedicated **Integration User** (a real **sys_user** record with no interactive login, holding only the role(s) this integration needs) via an **OAuth Application Registry** entry. This is the recommended pattern for a service integration — avoid using a real administrator's personal account.

1. Create the Integration User first: **User Administration** → **Users** → **New**. Give it a clear name (e.g. **crogl-chatops-integration**), no password requirement, and leave it non-interactive.
2. Assign it whatever role(s) grant read and write access to your chosen target table (see [Permissions to grant](#) below).

Permissions to grant

IMPORTANT

ServiceNow's role model is heavily customized per instance, and this connector's **send-verification/add-comment/close** writes need whatever role your instance actually requires to create and update records on the **specific** table your ChatOps flow watches — there is no single universal role name to cite here, the way there might be for a fixed, un-customized API surface. Confirm the correct role with whoever administers ACLs on that table (System Security → Access Control (ACL) → filter by your target table), rather than guessing. This has not been live-verified.

Separately from table-level ACLs, the Client Credentials grant type itself must be enabled instance-

wide before any OAuth Application Registry entry using it will work:

1. Navigate to **System Properties** (`sys_properties.list`).
2. Create or update the property named `glide.oauth.inbound.client.credential.grant_type.enabled`, type `true | false`, value `true`.

Then create the OAuth Application Registry entry:

1. Navigate to **System OAuth** → **Application Registry**, click **New**, and select "**Create an OAuth API endpoint for external clients**" from the interceptor list.
2. Fill in a name for the entry. Per ServiceNow's own guidance for this grant type, leave **Redirect URL** and any login-URL fields blank—they are not used by Client Credentials. Do **not** set **Public Client** to `true` (this grant type requires a confidential client per RFC 6749).
3. The **OAuth Application User** field—the field that maps this registry entry to the Integration User created above—is **not on the form by default**. Add it via **Form Builder** on the Application Registry record, then set it to the Integration User.

NOTE

The exact field layout and interceptor wording can vary slightly by ServiceNow release. This section is sourced from ServiceNow's own developer-blog walkthrough for the Washington DC release (see Reference documentation below), not a first-hand console check on a live instance—treat field labels as accurate-as-documented rather than independently re-verified.

Generating the credential

1. Save the Application Registry entry. ServiceNow auto-generates a **Client ID**; click the lock icon (or the equivalent "generate secret" control) next to the Client Secret field to generate and reveal the **Client Secret**. Copy both immediately—the secret is not retrievable in plaintext again after you navigate away.
2. Note your instance hostname (e.g. `mycompany.service-now.com`)—you will need it in the next step.
3. Decide which table this connector instance will target (per the Overview above)—this is entered directly into Crogl, not generated in ServiceNow.

Mapping values into Crogl

Crogl field	Where it came from	Notes
<code>host</code>	Your ServiceNow instance hostname	No scheme — e.g. <code>mycompany.service-now.com</code> , not <code>https://mycompany.service-now.com</code> .
<code>target_table</code>	The table your ChatOps/Now Actions delivery flow watches	Confirmed with your ServiceNow ChatOps owner, not generated by any setup step above. Must have an <code>assigned_to</code> field.

Crogl field	Where it came from	Notes
<code>client_id</code>	Auto-generated on the OAuth Application Registry entry	Visible on the entry's form after saving.
<code>client_secret</code>	Generated on the same entry (lock icon / "generate secret")	Copy at generation time — not retrievable again afterward. Never paste this into a chat conversation with Crogl; enter it directly into the connector's credential form.

Verifying the setup

Before creating the connector instance in Crogl, mint a token directly against ServiceNow to isolate credential problems from Crogl-side problems:

```
curl -s -X POST "https://<host>/oauth_token.do" \
  -d "grant_type=client_credentials" \
  -d "client_id=<client_id>" \
  -d "client_secret=<client_secret>"
```

- **A JSON body with `access_token/token_type: Bearer`** — the credential itself is good; any remaining failure once the connector is created is a table-permission problem (see below), not a credential problem.
- **400/401 from this call** — the Client ID/Secret pair is wrong, the Application Registry entry isn't saved correctly, or the `glide.oauth.inbound.client.credential.grant_type.enabled` property above isn't `true`. Not a Crogl-side issue — fix it here first.
- **A working token, but 403 on Crogl's first `send-verification` or `query` call once the connector is created** — the Integration User lacks read/write access to the target table. Distinguish this from the credential problem above: token minting succeeding means auth is fine; the 403 is happening one layer further in, at ServiceNow's ACL check on the specific table.
- **A working token and working `query`, but 403 only on `send-verification/add-comment/close`** — the Integration User has read but not write access on the target table; grant the write-side ACL/role specifically.

Reference documentation

- ServiceNow Community developer blog, "Up Your OAuth2.0 Game: Inbound Client Credentials with Washington": <https://www.servicenow.com/community/developer-blog/up-your-oauth-2-0-game-inbound-client-credentials-with-washington/ba-p/2816891> — source for the Client Credentials setup steps above (system property name, Application Registry interceptor flow, OAuth Application User field / Form Builder step).
- ServiceNow Now Support KB1645212, "How to configure Client Credentials grant type for inbound OAuth" — corroborating source for the same setup flow; behind a customer support login, not independently fetched in this session.

- ServiceNow Table API reference (Yokohama bundle): https://www.servicenow.com/docs/bundle/yokohama-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html — general Table API auth and request-shape reference.
- ServiceNow Community, "How to generate oauth token with client_credentials grant type using curl command" (Now Support KB3000226) — source for the `/oauth_token.do` request shape used in the verification `curl` command above.